Parallel Processing and Pipeline Hazards

Week 11 — Flynn's Taxonomy, Instruction-Level Parallelism, and Safe Throughput

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

September 5, 2026

Where Week 10 Left Us

One bottleneck kept changing shape

- Week 9 used locality to hide DRAM latency behind cache.
- Week 10 used pages, frames, and storage to give a process a larger, protected address space than physical memory alone could provide.
- Both designs ask the same question: **how can the machine keep useful work moving while one operation is slow?**

Today's turn

What if the processor itself has several useful operations ready at once? Parallel organizations and pipelines try to overlap those operations, but overlap is safe only when resources and dependencies permit it.

Motivation: One Instruction at a Time Wastes Hardware

A five-step instruction path

Suppose each instruction needs five stages: fetch, decode, execute, memory, and writeback. If one instruction occupies the whole path before the next starts, four stage resources sit idle most of the time.

- A **pipeline** overlaps different instructions in different stages.
- After the pipeline fills, one completed instruction can emerge each cycle even though each instruction still takes several stages of latency.
- The promise is **throughput**, not magic reduction of every instruction's individual latency.

This is the standard pipeline motivation in Hamacher Ch. 10 [1] and Stallings 11e, §16.4, PDF pp. 667–685 [2].

Roadmap: From Many Processors to One Safe Stream

1. Classify parallel machines by instruction streams and data streams.
2. Distinguish data-level, thread-level, and instruction-level parallelism.
3. Build a five-stage instruction pipeline around one running stream.
4. Diagnose structural, data, and control hazards.
5. Resolve hazards with scheduling, duplication, forwarding, stalls, and prediction.
6. Compare design choices: depth, clock period, bubbles, and complexity.

Running instruction stream

```
LW R1,0(R2); ADD R3,R1,R4; SUB R5,R3,R6; BEQ R5,R0,L; OR R7,R8,R9
```

First classify the kind of parallel work the machine exposes.

Flynn's Question: How Many Streams Are Moving?

The organizing idea

Classify a computer by the number of concurrent **instruction streams** and **data streams** it processes. This is a taxonomy of organization, not a ranking from "bad" to "good."

- **Instruction stream:** the sequence of operations being issued.
- **Data stream:** the operands or data elements those operations consume.
- A single physical chip can support more than one mode at different levels: for example, multiple cores may each run a SISD-like stream while a vector unit executes SIMD operations.

Flynn's taxonomy is standard course treatment from Hamacher Ch. 5 [1]; Stallings 11e's brief parallel-organization discussion is in §20.1 [2].

The Four Flynn Classes

Class	Instruction streams	Data streams	Typical interpretation
SISD	One	One	Conventional sequential core; one instruction acts on one data item
SIMD	One	Many	One operation applied across a vector or image row
MISD	Many	One	Rare and specialized; multiple operations inspect one stream
MIMD	Many	Many	Multicore, multiprocessor, or cluster with independent work

Exam discipline

Count streams, then justify with the work pattern. "Many ALUs" alone does not establish SIMD; the instruction stream must be shared.

Worked Classification: Three Workloads

Classify before naming the hardware

1. Add two 1024-element vectors using one vector instruction per chunk.
2. Run four independent matrix tiles on four cores.
3. Apply several independent filters to the same sensor stream.

- (1) is **SIMD**: one operation pattern, many data elements.
- (2) is **MIMD**: independent instruction streams and data regions.
- (3) resembles **MISD** only under a strict stream definition; in practice it is often implemented as MIMD workers sharing an input buffer.
- The taxonomy describes the organization; the workload decides whether that organization is useful.

Parallelism Has Three Different Scales

Do not collapse these into one word

- **Data-level parallelism (DLP):** the same operation over many elements, as in vector addition or pixels.
- **Thread-level parallelism (TLP):** independent instruction sequences execute on separate cores or hardware threads.
- **Instruction-level parallelism (ILP):** one instruction stream contains independent instructions that the processor may overlap.

- Flynn's classes describe streams; DLP/TLP/ILP describe where useful concurrency comes from.
- The rest of this deck focuses on **ILP inside one instruction stream**, where hazards are the price of overlap.

Socratic Check: Which Parallelism Is Present?

Scenario

A processor has four cores. Each core runs a different loop iteration, and each loop iteration contains a five-stage pipeline. Which kinds of parallelism are present?

- TLP: four cores execute independent instruction streams.
- ILP: each core overlaps instructions within its own stream.
- Not automatically SIMD: the cores may execute different instructions or different control paths.
- A good answer names the level and the evidence, rather than counting only the number of cores.

Now overlap one instruction stream without violating its meaning.

The Five-Stage Pipeline Model

A useful teaching model

IF	ID	EX	MEM	WB
Fetch	Decode/read	ALU/branch	Data memory	Register write

- Pipeline registers separate stages and hold intermediate results.
- Every stage advances on a clock edge; the slowest stage constrains the clock period.
- The same instruction visits all five stages, but different instructions occupy different stages simultaneously.

The stage names are a simplified course model; real processors may split, merge, repeat, or rename these functions.

Timing Example 1: Filling a Five-Stage Pipeline

Four independent instructions

Assume no hazards and one cycle per stage. Compare sequential execution with a five-stage pipeline.

	1	2	3	4	5	6	7	8
I_1	IF	ID	EX	MEM	WB			
I_2		IF	ID	EX	MEM	WB		
I_3			IF	ID	EX	MEM	WB	
I_4				IF	ID	EX	MEM	WB

- Sequential: $4 \times 5 = 20$ stage-cycles. Pipelined: $5 + (4 - 1) = 8$.
- Speedup here is $20/8 = 2.5$, below the ideal five because the stream is finite and the pipeline must fill and drain.

Latency, Throughput, and the Ideal Limit

Three quantities, three questions

- **Latency:** how long from an instruction's IF to its WB?
- **Throughput:** how often can completed instructions emerge?
- **Speedup:** how does total completion time compare with a non-overlapped baseline?

- With k balanced stages and n independent instructions, $T_{pipe} \approx k + (n-1)$ cycles; sequential time is nk cycles.
- For large n, ideal speedup approaches k, but only if stages are balanced and hazards, fills, drains, and overhead are negligible.
- A deeper pipeline can raise clock frequency while increasing branch penalty, register overhead, and recovery cost.

Pipeline Architecture: The Design Questions

1. **Stage balance:** can work be divided so no stage dominates the clock period?
2. **Pipeline registers:** what state must cross each boundary, and what setup/clock-to-q overhead is added?
3. **Resource replication:** can instruction fetch and data access happen together, or must they share one memory port?
4. **Dependence handling:** where can a value be forwarded, and when must the pipeline wait?
5. **Control recovery:** how many wrong-path instructions are flushed after a branch is resolved?

These are architecture choices, not merely implementation details: they determine CPI, area, power, and verification burden.

Socratic Check: What Does Pipelining Not Do?

Claim

"A five-stage pipeline makes every instruction take one cycle." Is that statement correct?

- No. An individual instruction still traverses five stages, so its latency is roughly five cycles in this model.
- The valid claim: after fill, independent instructions can complete at a rate of one per cycle.
- Stalls and flushes reduce the achieved throughput; they appear as bubbles or discarded work in the timing chart.

The pipeline is fast only when its assumptions remain true.

Hazard Taxonomy: Why Would Overlap Be Unsafe?

A hazard is a condition that prevents the next instruction from executing in its intended cycle.

- **Structural hazard:** two stages need the same hardware at once.
- **Data hazard:** an instruction depends on a value whose production or writeback has not completed.
- **Control hazard:** the next instruction address is uncertain, usually because of a branch or exception.

- The remedy must match the cause: add/duplicate a resource, move or forward data, or predict and recover from control flow.

Structural Hazard: One Port, Two Requests

Conflict in cycle 4

In a five-stage pipeline, I_1 is in MEM while I_2 is in IF. If one single-ported memory serves both instruction fetches and data accesses, both stages request the same port in the same cycle.

- **Option 1:** stall one instruction, inserting a bubble.
- **Option 2:** split instruction and data memories/caches, allowing the two accesses to proceed in parallel.
- **Option 3:** use multiported or time-multiplexed hardware, paying area, energy, or timing cost.
- A structural hazard is about *resource demand*, not whether the instructions compute related values.

Data Hazards: The Running Stream

RAW dependence

```
LW R1,0(R2)
ADD R3,R1,R4
SUB R5,R3,R6
```

- `ADD` needs the value produced by `LW`; this is a **read after write (RAW)** dependence.
- `SUB` needs the value produced by `ADD`; another RAW dependence chains the stream.
- **WAR** (write after read) and **WAW** (write after write) can arise when instructions execute or complete out of order; in a simple in-order five-stage pipeline, RAW is the central hazard.

Timing Example 2: A Load-Use Stall

Assumption

A load's data is available after MEM. The dependent ALU instruction needs it at the start of EX. One bubble is therefore required even with forwarding.

	1	2	3	4	5	6	7	8
`LW`	IF	ID	EX	MEM	WB			
`ADD`		IF	ID	**stall**	EX	MEM	WB	
`SUB`			IF	**stall**	ID	EX	MEM	WB

- The hazard unit freezes PC and IF/ID for one cycle and inserts a bubble into EX.
- The load's result can then be forwarded from the MEM/WB boundary to the dependent ADD's EX input.

Control Hazard: Which Instruction Is Next?

The branch changes the instruction stream

- The fetch unit may fetch the fall-through instruction before the branch condition is known.
- If the branch is taken, those fetched instructions are wrong-path work and must be prevented from changing architectural state.
- The penalty is the number of younger stages that must be flushed or redirected when the branch resolves.

- **Delay the decision:** simple, but inserts bubbles.
- **Predict and recover:** keep fetching; flush on a misprediction.
- **Move branch resolution earlier:** reduces penalty but lengthens an earlier stage and may add hardware to the critical path.

Socratic Check: Name the Hazard and the Cost

Three cases

1. One memory port is requested by IF and MEM in the same cycle.
2. An ADD reads a register before a preceding load produces it.
3. A conditional branch has not yet resolved its target.

- (1) **Structural**: duplicate, split, or schedule the resource.
- (2) **Data / RAW**: forward when possible, otherwise stall.
- (3) **Control**: predict, delay, or flush wrong-path work.
- A complete answer states both the category and the mechanism that pays for avoiding it.

Resolve the conflict at the cheapest point that preserves correctness.

Resolution 1: Stall and Bubble

The conservative mechanism

When the next instruction cannot safely advance, the hazard unit holds the blocked instructions and inserts a no-op bubble into the following stage.

- The program's meaning is preserved because the dependent operation waits for the required resource or value.
- A stall can freeze the PC, freeze an inter-stage register, and clear the control fields entering the next stage.
- The cost is visible as extra cycles: if a fraction s of instructions incur an average b bubbles, then approximately $\mathrm{CPI} = 1 + s \times b$ before other effects.

Resolution 2: Forward the Value Before Writeback

Why wait for the register file?

If an ALU result is already sitting at the EX/MEM boundary, the next ALU operation can consume it directly instead of waiting for WB to write and ID to read the register file.

- Comparators detect whether a source register matches a destination register in a later pipeline stage.
- Multiplexers select the newest available value for each ALU input.
- Forwarding removes many ALU-to-ALU RAW stalls, but it cannot remove a load-use delay when memory data arrives too late.

Forwarding/data bypassing is part of the indexed pipeline treatment in Stallings 11e, §16.4 and the RISC discussion in Ch. 17 [2].

Resolution 3: Schedule and Rename

- **Compiler scheduling:** move an independent instruction into a delay slot or between a producer and consumer when the ISA permits it.
- **Register renaming:** give independent writes different physical destinations, removing false WAR and WAW name conflicts.
- **Scoreboarding/reservation stations:** track operand readiness and resource availability so independent instructions can proceed.
- These techniques exploit more ILP, but require metadata, selection logic, recovery rules, and stronger verification.

Boundary

No scheduling trick can remove a true RAW dependence: the consumer still needs the producer's value.

Resolution 4: Predict, Then Recover

Control-flow speculation

A branch predictor guesses direction and sometimes target so fetch continues without waiting for full branch resolution.

- Correct prediction turns a control hazard into useful work.
- Wrong prediction requires a **flush**: invalidate younger instructions and restart from the correct target.
- Expected branch cost depends on branch frequency, misprediction rate, and recovery penalty:

$$\Delta\mathrm{CPI} \approx f_{br}\, p_{miss}\, P_{flush}.$$

- Deeper pipelines can increase P_{flush} even if their clock is faster.

Architecture Design: The Throughput Budget

Choice	Potential gain	New cost	Question to ask
Deeper pipeline	Shorter stage delay; higher clock	More registers, bubbles, branch penalty	Are hazards predictable enough?
Duplicate resources	Fewer structural conflicts	Area, power, coherence/consistency	Is the conflict frequent?
Forwarding network	Fewer RAW stalls	Comparators, muxes, timing paths	Can the value arrive in time?
Branch prediction	Higher control-flow throughput	Predictor state and flush recovery	What is the miss penalty?
Out-of-order issue	More ILP exposed	Rename, queues, precise recovery	Is added complexity worth it?

Design principle: maximize useful completed work per unit time, not merely the clock rate or the number of stages.

Running Stream: A Complete Hazard Diagnosis

Stream

`LW R1,0(R2); ADD R3,R1,R4; SUB R5,R3,R6; BEQ R5,R0,L; OR R7,R8,R9`

- `LW` → `ADD`: true RAW; forward plus one load-use stall in the five-stage timing model.
- `ADD` → `SUB`: true RAW; ALU forwarding can avoid an additional stall.
- `SUB` → `BEQ`: branch depends on a just-produced comparison value; forwarding or delayed branch resolution matters.
- `BEQ` → `OR`: control hazard; fetch may need a prediction and a flush if the branch is taken or mispredicted.

A timing diagram is a proof of the schedule: mark the producer stage, consumer stage, available bypass path, and every inserted bubble.

Socratic Check: Choose the Cheapest Correct Fix

Scenario

An ALU result is ready at the end of EX, its consumer enters EX next cycle, and the machine already has an ALU bypass path. What should the hazard unit do? What if the producer is a load instead?

- ALU producer: forward the result; no bubble is needed.
- Load producer: **stall until the data is available**, then forward; the memory access finishes too late for the immediately following EX.
- The correct resolution depends on *when* the value becomes available, not only on the fact that a dependence exists.

Week 11 Synthesis: A Decision Procedure

1. Identify the streams: instruction count and data count.
2. For a pipeline conflict, name the blocked stage and requested resource or operand.
3. Classify the hazard: structural, data, or control.
4. Ask whether duplication, forwarding, scheduling, prediction, or a stall resolves it without changing program meaning.
5. Count the bubbles or flushes and update the timing diagram.
6. State the design trade-off: throughput, latency, area, power, complexity, and recovery cost.

Bridge to Week 12

Next week turns these mechanisms into worked instruction-pipelining problems, then compares RISC and CISC choices that make hazards easier or harder to manage.

References and Source Boundaries

- Hamacher, Vranesic, Zaky, and Manjikian, *Computer Organization and Embedded Systems*, 6th ed., Chs. 5 and 10. Chapter-level course anchor; current repository index does not page-verify these chapters.
- Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. (repository key retained as Stallings2015), §16.4, "Instruction Pipelining," PDF pp. 667–685; §20.1, "Multiple Processor Organizations," PDF p. 847.
- Flynn's taxonomy is not attributed to Stallings here: its treatment in the indexed 11th edition is brief; the course anchor is Hamacher Ch. 5.

Textbook page numbers are used only where the repository's index marks them as checkable. Chapter citations remain chapter citations.

Bibliography I


V. Carl Hamacher, Zvonko G. Vranesic, Safwat G. Zaky, and Naraig Manjikian.
Computer Organization and Embedded Systems.
McGraw-Hill, 6th edition, 2012.
ISBN 978-0-07-338065-0. Bib key retained as Hamacher2002_computer_organization for citation-key stability across existing lectures; the file indexed under master_supporting_docs/CS401/supporting_books/Hamacher2002/ and page-cited by Week 8 is this 6th edition, not the 5th ed. (2002) the key name suggests – see that folder's index.md Edition notice, 2026-08-11.


William Stallings.
Computer Organization and Architecture: Designing for Performance.
Pearson, 10th edition, 2015.